

PROJECT REPORT

LEARNING :

I would divide it into these areas:

1. **NLP**

You learned:

Text preprocessing

Lowercasing and text cleaning

Tokenization

Stopword removal

Lemmatization

TF-IDF

Sentiment classification

2. **Machine Learning**

Train/validation/unseen-test splitting

Stratified splitting

Logistic Regression

Balanced Logistic Regression

Multinomial Naive Bayes

Linear SVM

Handling class imbalance

Model comparison

3. **Model Evaluation**

You learned how to use and interpret:

Accuracy

Precision

Recall

F1-score

Macro F1

Weighted F1

Classification reports

Class-level performance

4. Transformers and Deep Learning

What Transformers are

BERT and DistilBERT

Tokenizers

Input IDs

Attention masks

Fine-tuning

Sequence classification

Logits

Softmax probabilities

Confidence scores

Hugging Face Trainer

5. Practical Model Training

Setting training parameters

Training on CPU

Saving checkpoints

Resuming interrupted training

Saving and loading a trained model

Using a pretrained model instead of training a Transformer from scratch

6. **Application Development**

Streamlit

Building a user interface for an ML model

st.session_state

Prediction history

Loading a trained model into an application

Connecting model predictions to UI components

Creating a Plotly sentiment gauge

7. **Software Engineering**

Creating requirements.txt

Writing a README

Organizing a project

Managing Python environments

Debugging dependency problems

Git and GitHub

Git LFS

Managing large ML model files

Creating a clean repository

8. **Overall learning**

Raw Data

↓

Data Cleaning

↓

EDA

↓

Preprocessing

↓

Feature Engineering

↓

Traditional ML Models

↓

Model Evaluation

↓

Transformer / DistilBERT

↓

Fine-tuning

↓

Model Selection

↓

Save Model

↓

Streamlit Application

↓

GitHub Deployment

Through this project, I gained practical experience in the complete machine-learning and NLP development lifecycle, from data cleaning and exploratory analysis to model training, evaluation, Transformer fine-tuning, and application development. I learned how to handle real-world issues such as duplicate data, class imbalance, dependency errors, interrupted training, and large model files. I also gained practical experience with Hugging Face Transformers, DistilBERT, Streamlit, Git, GitHub, and Git LFS. Overall, the project helped me understand how an NLP model can be developed, evaluated, saved, and integrated into a functional application.

Problem Statement :

- Customer reviews contain valuable information about customers' opinions, satisfaction, and experiences. However, when a business receives a large number of reviews, manually reading and analyzing every review becomes time-consuming, inefficient, and difficult to scale.
- The purpose of this project is to develop an automated sentiment analysis system that can analyze customer reviews and classify them into three categories: Negative, Neutral, and Positive. The system uses Natural Language Processing and Machine Learning techniques to understand the text of a review and automatically determine its sentiment.
- The project also provides a confidence score and an interactive interface so that users can quickly understand the sentiment of individual customer reviews. This can help businesses analyze customer feedback more efficiently and identify overall customer sentiment without manually reviewing every piece of feedback.

In simple words

The problem is:

Thousands of customer reviews → humans cannot efficiently read everything → our system automatically analyzes them → Negative / Neutral / Positive.

For example:

- The product quality is excellent and I am very happy with my purchase.
- Instead of a person manually analyzing it, your system automatically produces:

Sentiment: Positive

Confidence: XX%

Challenges :

1. **Duplicate reviews**
2. **Class imbalance**
3. **Poor Neutral-class performance**
4. **Missing ML dependencies**
5. **CPU-based DistilBERT training**
6. **Laptop shutdown during training**
7. **Kernel restart/in-memory variable loss**
8. **Streamlit session-state/history errors**
9. **Large model file and Git LFS**
10. **GitHub repository/history cleanup**

Challenge - How it was resolved :

- 1 Duplicate Reviews in the Dataset** Identified duplicate Review_text values and removed them using drop_duplicates(). This reduced the dataset to approximately 49,877 unique reviews.
- 2 Class Imbalance** Used stratified splitting to preserve class proportions and tested Balanced Logistic Regression using class_weight="balanced". Macro F1 was used alongside accuracy to evaluate class balance.
- 3 Poor Neutral-Class Performance** Examined class-level precision, recall and F1 rather than relying only on accuracy. Multiple models were compared, and the Neutral class was identified as an area requiring future improvement.
- 4 Missing/Conflicting ML Dependencies** Installed and configured the required packages such as PyTorch, Datasets, PyArrow and Accelerate, then verified the environment before continuing training.
- 5 Slow DistilBERT Training on CPU** Configured DistilBERT for CPU execution, used a small batch size and limited the maximum sequence length to 128 tokens to make training manageable.
- 6 Training Interrupted by Laptop Shutdown** Used the Hugging Face Trainer checkpoint (checkpoint-9825) and resumed training from the checkpoint instead of restarting the entire training process.
- 7 Saving and Loading the Trained Model** Saved the final DistilBERT model and tokenizer in distilbert_sentiment_final, allowing the Streamlit application to load the trained model without retraining.
- 8 Streamlit Session-State and History Errors** Initialized st.session_state variables properly and standardized the history dictionary keys such as Review, Sentiment, and Confidence.
- 9 Creating a Dynamic Sentiment Gauge** Used Plotly to create a custom Negative/Neutral/Positive gauge and connected its needle position to the actual model prediction.
- 10 Large Model File and GitHub Management** Used Git LFS for the large model.safetensors file and .gitignore to exclude unnecessary checkpoints. A clean Git history was then created and pushed to GitHub.

Data → ML → Transformer → Training → Application → Deployment